Leonardo Naime Lima

Analisador Léxico para a Linguagem C-: Projeto de Implementação utilizando Máquina de Moore

Relatório técnico do projeto solicitado pelo professor Rogerio Aparecido Gonçalves na disciplina de Teoria da computação do Bacharelado em Ciência da Computação da Universidade Tecnológica Federal do Paraná.

Universidade Tecnológica Federal do Paraná - UTFPR

Departamento Acadêmico de Computação - DACOM

Bacharelado em Ciência da Computação - BCC

Campo Mourão

Fevereiro / 2025

Resumo

Este relatório descreve os avanços e os resultados obtidos na atividade prática realizada, cujo propósito foi a concepção e implementação de um analisador léxico para a linguagem C-, utilizando uma máquina de Moore e a linguagem de programação Python.

Sumário

1 Introdução 4
2 Objetivos 4
3 Fundamentação 4
4 Materiais 4
5 Resultados Esperados 4
6 Instruções 5
7 Análise Léxica 5
8 Análise Sintática 6
9 Análise Semântica 6
10 Código 6
11 Referências 7

1 Introdução

Antes da execução de um programa, o compilador precisa interpretar seu código-fonte e convertê-lo em tokens, unidades sintáticas fundamentais. Este trabalho tem como objetivo desenvolver um analisador léxico para a linguagem C-, uma versão simplificada de C, empregando autômatos finitos. O analisador será capaz de identificar tokens como palavras-chave, operadores, identificadores e números, organizando-os em uma estrutura processável.

A implementação utilizará uma Máquina de Moore, que define saídas com base nos estados e transições. Expressões regulares serão empregadas para especificar padrões da linguagem, convertendo-os em um autômato finito determinístico. Dessa forma, este projeto estabelece a base para a primeira etapa de um compilador para C-.

2 Objetivos

Neste projeto, será desenvolvida uma ferramenta para análise léxica da linguagem C-, baseada em autômatos finitos e máquinas de Moore. O analisador léxico terá a função de extrair e categorizar tokens do código-fonte, produzindo uma lista estruturada que servirá como entrada para as etapas seguintes do compilador, como a análise sintática e semântica.

3 Fundamentação

A elaboração deste trabalho teve como referência as diretrizes, o conceito do automato inicial e do código base disponibilizados pelo professor.

4 Materiais

- Kernel Linux 6.11.3 LTS;
- Linux Mint 21 Cinnamon;
- Computador com processador intel I5-10400F, memória RAM 16 GB;
- Visual Studio Code (Python) e JFLAP;

5 Resultados Esperados

O analisador léxico deve ser capaz de interpretar um código-fonte escrito em C- e produzir uma lista precisa de tokens. A ferramenta precisa operar de forma eficiente,

reconhecendo corretamente palavras-chave, operadores, identificadores e números, mesmo em diferentes combinações dentro do código.

6 Instruções

Para executar o código, utilize o seguinte comando:

```
pytest
```

Caso queira mais detalhes sobre a execução, utilize a opção verbose:

```
pytest -v
```

Para executar um teste específico e ver seu token, utilize o comando

```
python3 analex.py -k tests/NOME_DO_TESTE
```

7 Análise Léxica

A primeira fase do processo de compilação é a análise léxica, responsável por transformar o código-fonte em uma sequência estruturada de tokens.

O analisador léxico percorre o código caractere por caractere, detectando padrões com base em expressões regulares. Para cada padrão identificado, um token correspondente é criado e encaminhado para a fase de análise sintática. Esse procedimento é dividido em duas etapas principais:

- **Leitura do código-fonte**: O código é percorrido sequencialmente, identificando possíveis padrões léxicos;
- **Reconhecimento de tokens**: Expressões regulares são aplicadas para extrair tokens válidos, como palavras-chave e operadores. Além disso, nessa etapa também são realizadas verificações quanto à existência do arquivo de entrada e tratamento de erros de uso.

O analisador léxico foi desenvolvido com base em uma máquina de Moore, implementada no arquivo `analex.py`.

8 Análise Sintática

A análise sintática representa a segunda etapa do processo de compilação. Seu papel é receber a sequência de tokens gerada pela análise léxica e determinar se essa sequência está estruturada conforme as regras sintáticas da linguagem de programação em questão.

Para realizar essa verificação, o analisador sintático organiza os tokens em uma estrutura hierárquica, geralmente representada por uma árvore sintática abstrata (AST).

9 Análise Semântica

A análise semântica constitui a terceira fase do processo de compilação. Seu objetivo é garantir que o código-fonte possua um significado coerente dentro das regras da linguagem de programação utilizada. Diferentemente da análise sintática, que verifica apenas a estrutura do código, a análise semântica se concentra no significado das construções presentes no código.

Para atingir esse objetivo, a análise semântica realiza diversas verificações, tais como:

- **Compatibilidade de tipos**: Verifica se as operações realizadas entre variáveis são compatíveis com seus respectivos tipos;
- **Declaração e uso de identificadores**: Confirma se variáveis e funções foram declaradas antes de seu uso, evitando referências indefinidas;
- **Regras de escopo**: Assegura que identificadores utilizados estão dentro de um contexto válido;
- **Controle de fluxo**: Avalia se estruturas como loops e condicionais são usadas corretamente dentro do código.

10 Código

O Analisador Léxico foi desenvolvido utilizando uma Máquina de Moore, implementada dentro do arquivo `analex.py`. Essa máquina possui múltiplos estados, nomeados de acordo com suas respectivas funções no código, garantindo uma melhor organização e clareza no processamento das entradas.

A estrutura do analisador é composta por:

- **Estados**: Definidos conforme suas funções específicas, permitindo um fluxo organizado de processamento;
- **Alfabeto**: Conjunto de caracteres reconhecidos, incluindo letras, números, operadores e símbolos especiais;
- **Tokens**: Unidades léxicas identificadas, como palavras reservadas, identificadores, números e operadores;
- **Transições**: Regras que determinam como a máquina muda de estado com base nos caracteres lidos, possibilitando o reconhecimento correto dos padrões do código-fonte.

Para validar a funcionalidade do Analisador Léxico, foi criado o arquivo `analex_test.py`, contendo testes automatizados com a biblioteca `pytest`. Esses testes verificam se as saídas do analisador correspondem às esperadas, armazenadas em arquivos de referência.

A execução dos testes pode ser realizada de duas formas:

- **Execução padrão**: Testa todos os arquivos automaticamente com o comando:

```
pytest
```

- **Execução detalhada**: Exibe informações adicionais sobre os testes com o comando:

```
pytest -v
```

Durante a fase de testes, foram identificadas inconsistências em alguns arquivos de saída esperada, exigindo ajustes para garantir que os testes reflitam corretamente o comportamento do analisador. Após essas correções, a execução dos testes ocorreu sem problemas, assegurando o funcionamento adequado do analisador.

11 Referências

RODGER, Susan H. *JFLAP: An Interactive Formal Languages and Automata Package*. Sudbury, MA: Jones & Bartlett Learning, 2006. ISBN 9780763738347.

LOUDEN, Kenneth C. *Compiladores: Princípios e Práticas*. São Paulo, SP: Thomson, 2004. ISBN 8522104220.

SIPSER, Michael. *Introdução à Teoria da Computação*. São Paulo, SP: Cengage Learning, 2007. ISBN 9788522104997.

MENEZES, Paulo Blauth. *Linguagens Formais e Autômatos*. Porto Alegre, RS: Bookman, 2011. ISBN 9788577808403.